# MEMORIA DEL PROYECTO DE APRENDIZAJE AUTOMÁTICO Y BIG DATA




CURSO 2025-2026

AUTOR
MARLON CHIRINO MIJENES

FACULTAD DE INFORMÁTICA - UNIVERSIDAD COMPLUTENSE DE MADRID

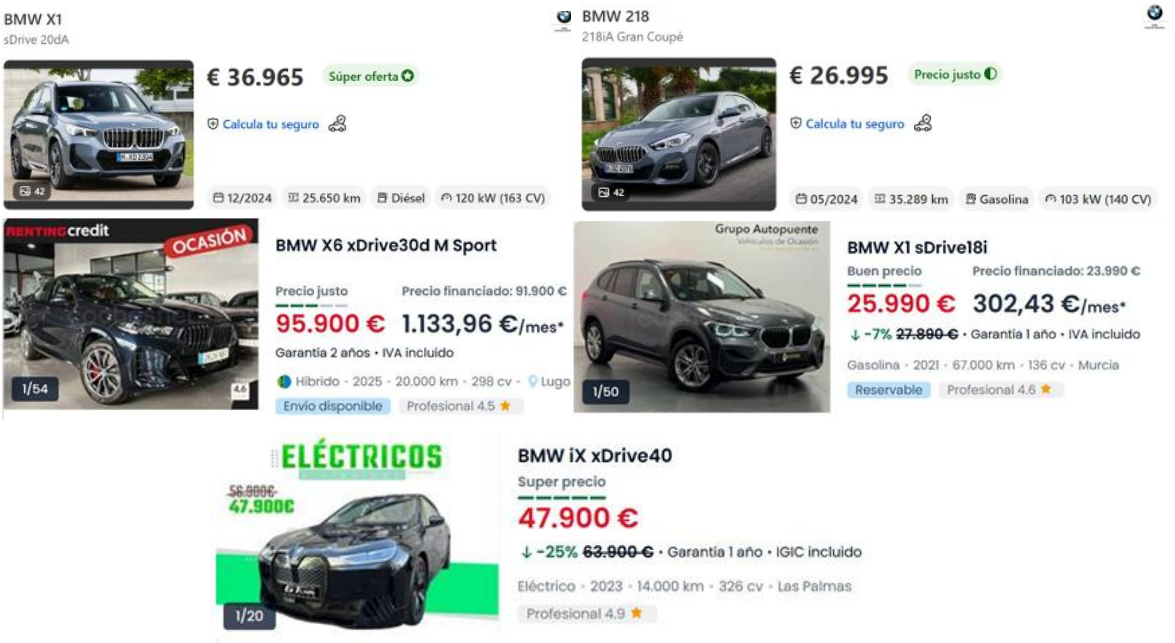
# TABLA DE CONTENIDO

Diseño de una red neuronal para un problema de clasificación ..... 3
1.Introducción y descripción del problema ..... 3
2.Objetivos del proyecto ..... 3
3.Descripción y análisis del conjunto de datos ..... 4
4.Preprocesado y limpieza de datos realizados ..... 4
5.Técnicas y modelos aplicados ..... 5
6.Metodología experimental ..... 6
7.Métricas utilizadas para la evaluación ..... 7
8.Resultados obtenidos ..... 7
9.Comparativa y análisis crítico de resultados ..... 8
10.Limitaciones del trabajo y posibles mejoras futuras ..... 9

# DISEÑO DE UNA RED NEURONAL PARA UN PROBLEMA DE CLASIFICACIÓN

## 1.INTRODUCCIÓN Y DESCRIPCIÓN DEL PROBLEMA

En el mercado de automóviles de segunda mano el precio depende de múltiples factores, así como los kilómetros, año, modelo, marca y potencia entre otros. En algunas de las páginas web más conocidas, como AutoScout24 o Coches.net, los anuncios se etiquetan de manera visual con un indicador que describe si el precio está en un buen rango.




Este proyecto tiene como fin replicar este funcionamiento utilizando Redes Neuronales implementadas en NumPy. Utilizaremos un conjunto de datos reales de coches de la marca BMW, el sistema evaluará las características de cada coche y lo clasificará en una de estas 3 categorías: “Ganga”, “Precio Justo” o “Caro”.

## 2.OBJETIVOS DEL PROYECTO

Los objetivos principales de este proyecto son varios. Buscamos construir un pipeline completo de Aprendizaje automático aplicado a un problema real. También implementar y entrenar una red neuronal MLP (Multi-Layer Perceptron) desde cero usando la librería NumPy. Posteriormente ajustaremos y optimizaremos nuestra red modificando hiperparámetros, maximizando el rendimiento de nuestra red. Por último, compararemos el rendimiento de la red neuronal creada frente a modelos ya disponibles en la librería SciKit-Learn como KNN.

## 3.DESCRIPCIÓN Y ANÁLISIS DEL CONJUNTO DE DATOS

El dataset seleccionado es *"BMW Used Car Dataset"* de Kaggle, que originalmente consta de unos 10.800 registros de ventas. Teniendo en cuenta que hay algunos modelos de coche de los cuales no hay suficientes ejemplares e intentando reducir el tiempo de ejecución, el conjunto se ha filtrado. Nos hemos quedado solo con los Serie 1 y Serie 3, dos de los modelos de los que más información tenemos, dejando el conjunto en 4.412 muestras.

Dado que el precio es una variable continua y este trabajo tiene como objetivo la clasificación y no la regresión, he aplicado una formula para generar las etiquetas objetivo. Primero se agruparon los vehículos por modelo y año de producción para calcular la mediana de precio del mercado. A continuación, calculé el ratio entre el precio de cada anuncio y la mediana de su segmento. En base al resultado, cada anuncio se etiquetó en una de las siguientes categorías:

- Ganga: Precio inferior al 85% de la mediana del mercado. Aquí encontramos el 18% de las muestras, que suman 813.
- Precio Justo: Precio entre el 85% y el 115% de la mediana, la gran mayoría de las muestras se encuentran aquí, 58%. En total tenemos 2550 muestras.
- Caro: Precio por encima del 115% de la mediana. Hay un 24% de vehículos caros, lo que representa 1049 muestras.

La distribución que obtenemos es muy realista. Por último, para evitar el data leakage, eliminaremos la variable original del precio exacto del conjunto de datos.

## 4.PREPROCESADO Y LIMPIEZA DE DATOS REALIZADOS

Para preparar los datos de entrada fue necesario aplicar varias transformaciones. Se separaron los datos en conjuntos de Train (80%) y Test (20%), siguiendo el principio Hold-Out Validation. Esto nos dejó con 3.529 muestras para el Train y las 883 muestras restantes para el Test. Dado que existe un desbalance en las muestras debido a que las gangas son minoría, la separación se hizo con un muestreo estratificado para mantener la proporción igual en ambos conjuntos.

El dataset contenía variables categóricas como el cambio manual o automático o el tipo de combustible. Estas fueron procesadas mediante One-Hot Encoding. Además, las variables numéricas presentaban un gran desbalance de escalas, como por ejemplo los kilómetros que son miles en comparación con el tamaño del motor que suele estar entre 1 litro para los Serie 1 menos potentes y 3 litros para los Serie 3 de más alta gama. Como las redes neuronales son sensibles a esto, las variables fueron estandarizadas usando StandardScaler para que todas pasaran a tener una media de 0 y una varianza de 1.

## 5.TÉCNICAS Y MODELOS APLICADOS

A diferencia de las prácticas de laboratorio, donde teníamos la matriz de pesos ya entrenada, en este proyecto la red neuronal fue implementada desde cero. Así que para romper la simetría inicial de la red se inicializó de manera al azar la matriz de pesos. Por lo tanto, los resultados de accuracy, así como la matriz de confusión y la tabla del classification report pueden variar en cada ejecución del código a pesar de mantener todos los hiperparámetros iguales.

Se ha utilizado un perceptrón multicapa de 3 capas, como se ha visto en clase. La capa de entrada está compuesta por 11 neuronas, cada una corresponde a una de las características técnicas de los vehículos tras el One-Hot Encoding y el escalado. La capa oculta consta de 35 neuronas, este fue el valor más equilibrado tras una serie de pruebas que se exponen en el siguiente punto, siendo capaz de capturar relaciones no lineales complejas (como la devaluación asociada al kilometraje) sin llegar a saturar el algoritmo. La capa de salida tiene solamente 3 neuronas, una para cada posible categoría definida.

Para el flujo de información se implementó ForwardPropagation como vimos en clase. En cada capa, el sistema calcula la combinación lineal de las entradas y sus pesos asociados, añadiendo en cada uno un "bias" o sesgo. Para acotar las salidas se emplea la función sigmoide, su fórmula es la vista en practicas a lo largo del curso:

$$g(z) = \frac{1}{1 + e^{-z}}$$

Para medir el error del modelo durante el aprendizaje se ha utilizado una función de coste similar a la vista en clase, con entropía cruzada multiclase regularizada. Para evitar que la red memorizase el Train, se incorporó un factor de regularización penalizando los pesos elevados en las matrices de transferencia. Después de varias pruebas expuestas a continuación, se vio que lambda = 0.1 introducía el ruido necesario en la optimización para mitigar el overfitting y forzar a la red a buscar patrones generales y reproducibles.

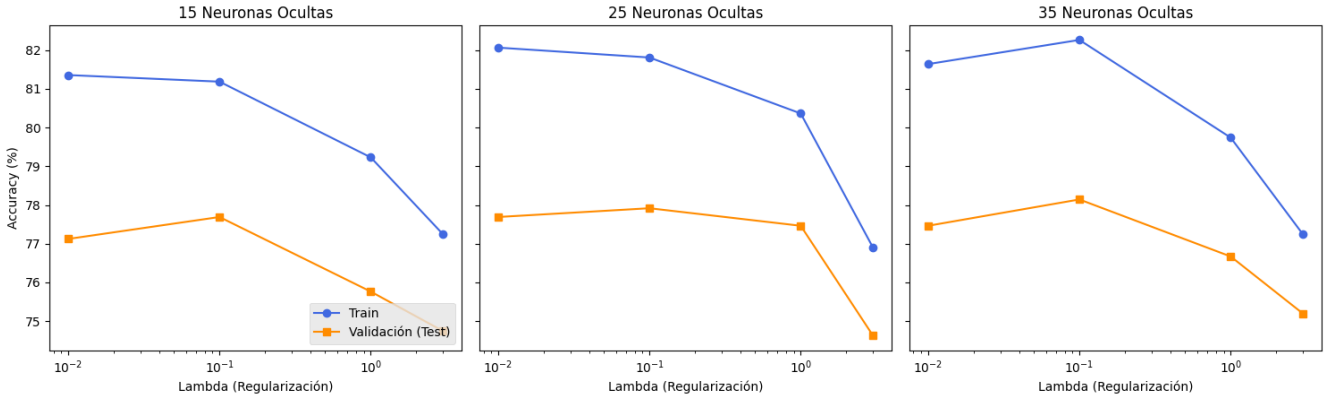
$$J(\Theta) = -\frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{K}\left[y_k^{(i)}\log((h_\Theta(x^{(i)}))_k) + (1-y_k^{(i)})\log(1-(h_\Theta(x^{(i)}))_k)\right] + \frac{\lambda}{2m}\sum_{l=1}^{L-1}\sum_{i=1}^{s_l}\sum_{j=1}^{s_{l+1}}(\Theta_{j,i}^{(l)})^2$$

Para la actualización y sincronización de los pesos se ha usado el algoritmo BackPropagation. Este calcula de manera analítica y recursiva de la salida hacia la entrada las derivadas parciales de la función de coste respecto a cada uno de los pesos de la red. En lugar de utilizar un gradient descent básico que podría estancarse en mínimos locales, estos gradientes calculados en NumPy se los pasamos a minimize de la librería SciPy, configurada con CG (Gradientes Conjugados).

Por último, se ha utilizado un modelo baseline basado en el algoritmo KNN (K-Nearest Neighbours) de la librería SciKit-Learn para comparar rendimientos.

## 6.METODOLOGÍA EXPERIMENTAL

Para encontrar los hiperparámetros perfectos se hicieron varias ejecuciones con diferentes cantidades de neuronas (15, 25, 35) y diferentes lambdas (0.01, 0.1, 1.0, 3.0). Esto se puede encontrar en el archivo busqueda_params.py. Dado el alto coste computacional de entrenar 12 modelos completos desde cero, en la fase de exploración espacial nos limitaremos a ejecutar cada prueba con 300 iteraciones. Esta técnica permite identificar las tendencias de aprendizaje y aislar las arquitecturas más prometedoras de forma eficiente. Una vez identificada la configuración ganadora (35 neuronas y lambda = 0.1), esta se entrenó de forma aislada hasta su convergencia total (1.000 iteraciones) para extraer las métricas definitivas del sistema.




Para auditar y validar la convergencia del modelo final seleccionado (35 neuronas en la capa oculta y lambda = 0.1), ejecutamos el algoritmo monitorizando su rendimiento exacto a lo largo de las iteraciones del optimizador de SciPy. Los datos extraídos fueron los siguientes:

- **300 iteraciones:** Train = 81.64% | Test = 78.14%
- **500 iteraciones:** Train = 82.94% | Test = 78.60%
- **1.000 iteraciones:** Train = 85.86% | Test = 79.28%
- **2.000 iteraciones:** Train = 86.06% | Test = 78.14%

Con estos datos podemos ver claramente el comportamiento de la curva de aprendizaje de la red y el inicio del sobreajuste. Durante las primeras 500 iteraciones, el modelo experimenta un aprendizaje acelerado y sano, mejorando significativamente su capacidad para generalizar datos nuevos. Sin embargo, a partir de ese punto entra en juego la ley de rendimientos decrecientes: empujar la red desde las 500 hasta las 1.000 iteraciones incrementa la precisión en el conjunto de entrenamiento en un notable 3%, mientras que la mejora real en el conjunto de Test es 0.4%. Al subir a 2.000 iteraciones nos damos cuenta de que ya habíamos tocado techo y que comenzamos a sufrir overfitting, ya que aumenta el rendimiento en el Train pero ya no lo hace en el Test, de hecho, disminuye.

Estos datos demuestran que el algoritmo de optimización ha extraído prácticamente todo el conocimiento útil del dataset en la frontera de las 1.000 iteraciones, marcando este límite como el techo técnico absoluto del modelo. Prolongar el entrenamiento más allá de este umbral solo serviría para degradar la capacidad de generalización del sistema en el mercado real.

## 7.MÉTRICAS UTILIZADAS PARA LA EVALUACIÓN

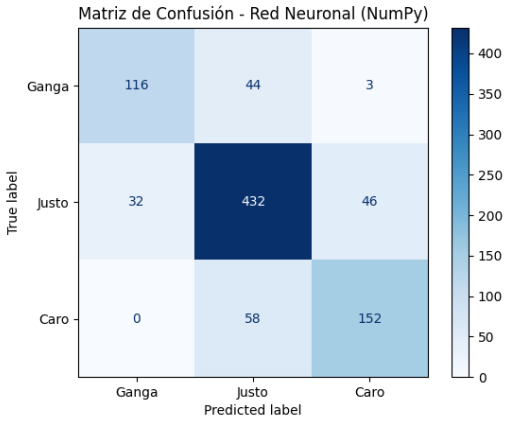
En la evaluación del modelo como se puede ver se han utilizado varias métricas. Para elegir qué modelo era el mejor se ha utilizado el Accuracy, con esta hemos construido las gráficas con las curvas de validación que aparecen en la anterior página. Mas adelante para evaluar el rendimiento del modelo elegido y compararlo con el K-NN también se ha usado el classification report, que nos aporta además de Accuracy, Precisión, Recall y F1-Score. Por último, para ayudar a que la representación de la solución fuera más visual, se ha añadido la matriz de confusión.

## 8.RESULTADOS OBTENIDOS

Tras entrenar nuestra red neuronal con la configuración que se ha ido mencionando a lo largo de la memoria (35 neuronas en la capa oculta, lambda = 0.1 y 1.000 iteraciones) los resultados de Accuracy fueron los siguientes:

- Accuracy en Train: 85.86%
- Accuracy en Test: 79.28%

Posteriormente se muestra la siguiente matriz de confusión:




De esta matriz se pueden sacar varias conclusiones. La primera de ellas es la alta sensibilidad en la clase mayoritaria, “Precio Justo”. Esta categoría la clasifica con alta fiabilidad, nuestra red tiene algunos errores, seguramente debidos a que son casos que se encuentran en la “frontera” de decisión. Como por ejemplo, confundir “Ganga” con “Precio Justo” o “Caro” con “Precio Justo”. Por otro lado, el modelo carece de erorres críticos, ya que en solo 3 ocasiones confunde una “Ganga” con un vehículo que tiene asignado un precio “Caro” y en el caso de vehiculos caros, su equivocación catalogándolos como gangas es 0.

Además, en el Classification Report obtuvimos los siguiente resultados. Dado el desbalanceo del mercado automovilístico en este conjunto de datos, la métrica más robusta para evaluar la relación real entre precisión y exhaustividad y en la que más nos fijaremos será el F1-Score.

Los resultados completos se sintetizan en la siguiente tabla:

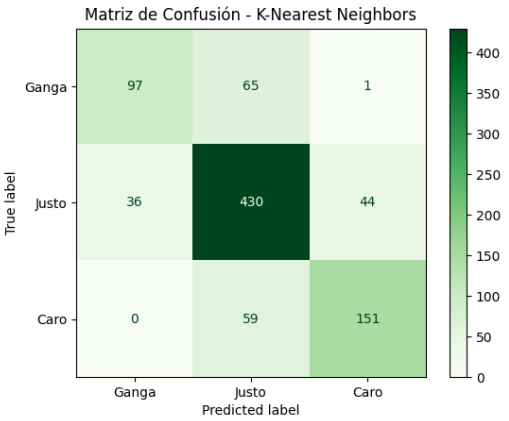
| | Precision | Recall | F1-Score | Support |
|---|---|---|---|---|
| **Ganga** | 0.78 | 0.71 | 0.75 | 163 |
| **Justo** | 0.81 | 0.85 | 0.83 | 510 |
| **Caro** | 0.76 | 0.72 | 0.74 | 210 |
| | | | | |
| **Accuracy** | | | 0.79 | 883 |
| **Macro Avg** | 0.78 | 0.76 | 0.77 | 883 |
| **Weighted Avg** | 0.79 | 0.79 | 0.79 | 883 |

## 9.COMPARATIVA Y ANÁLISIS CRÍTICO DE RESULTADOS

Para evaluar la viabilidad de la implementación de nuestra red neuronal con NumPy, he decidido hacer una comparación entrenando un algoritmo K-Nearest Neighbors (K-NN) de la librería SciKit-Learn. Este fue entrenado con k = 5 vecinos y bajo las mismas condiciones de preprocesamiento de datos, usando One-Hot Encoding y StandardScaler. Adjunto los resultados mencionados anteriormente de nuestra red y los de K-NN para que la comparación sea más "visual":

- Red Neuronal: Train = 85.86% y Test = 79.28%
- K-NN: Train = 84.10% y Test = 76.78%

Gracias a estos podemos extraer dos conclusiones. En primer lugar, el modelo K-NN sufre un overfitting evidente ya que alcanza un accuracy en train de 84% mientras que en Test decae a un 76%. Cae un 7.3% mientras que nuestra red neuronal cae un 6.5%. Por lo tanto, nuestra generalización es mucho más robusta, lo que demuestra que el ajuste de los hiperparámetros ha sido un éxito. En segundo lugar, podemos ver el poder de las arquitecturas multicapa, ya que como las capas ocultas han sido capaces de extraer relaciones no lineales entre características, se ha podido superar al algoritmo de la librería, que ya está muy optimizado.




Además, en el Clasification Report obtuvimos los siguiente resultados:

| | Precision | Recall | F1-Score | Support |
|---|---|---|---|---|
| **Ganga** | 0.73 | 0.60 | 0.66 | 163 |
| **Justo** | 0.78 | 0.84 | 0.81 | 510 |
| **Caro** | 0.77 | 0.72 | 0.74 | 210 |
| | | | | |
| **Accuracy** | | | 0.77 | 883 |
| **Macro Avg** | 0.76 | 0.72 | 0.74 | 883 |
| **Weighted Avg** | 0.77 | 0.77 | 0.76 | 883 |

Estas métricas nos confirman la superioridad técnica del Perceptrón Multicapa implementado frente al algoritmo de proximidad espacial (K-NN). La Red Neuronal supera al K-NN en el F1-Score de absolutamente todas las categorías de precio. Especialmente en la clase minoritaria "Ganga", donde logra una mejora de 3 puntos porcentuales, por lo tanto, tiene una mayor capacidad para detectar los verdaderos chollos del mercado.

En conclusión, la red neuronal, además de cumplir con el requisito matemático del proyecto, se perfila como un modelo matemáticamente más apto y seguro para su hipotético despliegue en una plataforma de compra y venta de vehículos.

## 10.LIMITACIONES DEL TRABAJO Y POSIBLES MEJORAS FUTURAS

El modelo ha demostrado una gran capacidad de generalización (superando el 78% de precisión en test sin overfitting), pero presenta ciertas limitaciones. Como hemos filtrado el dataset para utilizar únicamente los modelos Serie 1 y Serie 3, el modelo no está preparado actualmente para evaluar vehículos de gamas superiores, como los Serie 7, o distintas como la de los SUV (Sport Utility Vehicle) Serie X. En estos la depreciación y las fluctuaciones de precio siguen distribuciones matemáticas diferentes. Por ejemplo, en el caso de los de gama premium, con una depreciación muy grande en los primeros años, que luego se estabiliza y posteriormente al cumplir unos 10 o 15 años de antigüedad se vuelve a acentuar debido a sus altos costes de mantenimiento. Tampoco hemos tenido en cuenta vehículos de la Serie M, que son modelos deportivos de alto rendimiento. Estos sufren depreciación en sus primeros años y posteriormente se revalorizan al ser los más buscados por los coleccionistas, jugando un gran papel en su precio el modelo especifico y los extras.

Como mejora futura, sería posible implementar una arquitectura de red más profunda y escalar el entrenamiento mediante procesamiento en GPU, lo que permitiría ingerir el dataset completo de BMW y añadir variables macroeconómicas temporales que afectan al mercado de segunda mano. Por ejemplo, en épocas con precios del combustible elevados, vehículos con gran potencia o motores grandes suelen ver su precio algo reducido. Además, hay que añadir que los datos de los mercados de segunda mano están fuertemente sesgados al país en el que se recogen los datos, ya sea por las preferencias de los ciudadanos o por la situación económica del propio. En Alemania, por extraño que parezca, se compran y venden más descapotables que en España, por lo tanto, están más cotizados. En Reino Unido los coches son más baratos que aquí debido a los elevados precios de los seguros, que pueden llegar a las cinco mil libras anuales. En España han tenido un gran efecto las limitaciones medioambientales para acceder a las grandes ciudades, elevando el coste de los coches más asequibles con etiquetas “C” o “ECO” y desplomando el precio de los coches sin etiqueta.